

Wronskian Solutions of (Supersymmetric) Bethe Ansatz Equations: Formalism and a Bertini-Based Numerical Continuation Pipeline

Zuxian He

Department of Physics, Uppsala University

September 2026

Abstract

We give a formal account of the mathematical construction and numerical realization implemented in the `BAE_Bertini` codebase, which computes Bethe roots of a nested integrable spin chain for eigenstates labeled by standard Young tableaux (SYT). The construction proceeds through a *Q-system*: a lattice of auxiliary polynomials $Q_{a,s}(u)$ obeying a discrete bilinear (Hirota/QQ) functional relation, solved not directly but by representing the $Q_{a,s}$ as Wronskian (Casoratian) determinants of a common set of elementary building blocks — a discrete analogue of the classical fact that Wronskians of solutions of a shared linear equation obey simple conservation laws. We derive the degree counting of the Q-functions from the geometry of the Young diagram, present the supersymmetric (bosonic/fermionic) generalization of the Wronskian solution obtained by choosing a *marked point* (equivalently a Kac–Dynkin grading) on the diagram, and derive the vacancy-number combinatorics of the associated rigged-configuration / string-hypothesis description. We then formalize the numerical strategy: a single chain inhomogeneity is treated as a deformation parameter Λ , and the Q-system is solved by a *parameter homotopy continuation* from an asymptotically free ($\Lambda \rightarrow \infty$) point, where the solution is combinatorially exact, down to the physical, homogeneous point $\Lambda = 0$. We recall the general theory of homotopy continuation (the Davidenko differential equation and its predictor-corrector discretization) and describe how the project delegates this step to the numerical algebraic geometry package **Bertini**, through an adaptive multiple-precision path tracker, invoked via a parameter-homotopy driver written in Python. We close with the combinatorial symmetry used to validate numerical solutions (a rigging-complementation involution, realized combinatorially as Schützenberger evacuation, under which Bethe roots are conjectured to negate), quantitative validation and performance results, and a discussion of the construction’s scope and limitations.

Contents

1	Introduction	3
1.1	Physical setting	3
1.2	Numerical strategy	3
1.3	Scope and evidentiary status of this report	3
1.4	Outline	4
2	The Q-System Formalism	4

2.1	Q-functions on the Young diagram	5
2.2	The Hirota (QQ) equation	5
3	The Supersymmetric Wronskian and the Marked-Point Freedom	7
3.1	Marked points and gradings	7
3.2	The super-Wronskian determinant	8
3.3	Two coefficient bases	8
4	The Deformation Parameter and the Sequential Construction	9
4.1	The chain inhomogeneity as a homotopy parameter	9
4.2	Asymptotic solution at large Λ	9
4.3	Sequential construction	9
4.4	Extraction of Bethe roots	10
5	Rigged Configurations, the String Hypothesis, and a Flip Symmetry	10
5.1	Strings and vacancy numbers	10
5.2	The Kerov–Kirillov–Reshetikhin bijection	11
5.3	A flip involution and a conjectured root symmetry	12
6	Numerical Continuation: Formalism and Realization	12
6.1	A concrete warm-up example	12
6.2	Homotopy continuation and the Davidenko equation	12
6.3	Specialization: a linear parameter homotopy in Λ	13
6.4	Numerical safeguards	13
6.5	A path-tracking failure mode: branch jumps	14
7	A Fully Worked Example: the Shape $(2, 1)$	14
8	Software Realization	15
9	Validation and Numerical Results	16
9.1	Regression testing	16
9.2	Validation of Conjecture 5.1: the flip check	16
9.3	Performance profile of a single tableau	17
10	Discussion	17
10.1	Summary	17
10.2	Limitations	18
10.3	Outlook	18

1 Introduction

1.1 Physical setting

Integrable spin chains of rank higher than one admit a *nested* Bethe ansatz: magnon excitations are organized into several coupled families of rapidities, one family per node of an auxiliary root system, and the full spectrum is obtained by solving a coupled system of Bethe ansatz equations (BAE) across all families simultaneously. For a chain of length L , the spectrum organizes into multiplets labeled by partitions λ of L (Young diagrams); within a multiplet of shape λ , individual eigenstates are labeled by the standard Young tableaux (SYT) of that shape — fillings of the diagram’s L boxes with the integers $1, \dots, L$ that increase along every row and down every column. This report is a self-contained account of a computational pipeline that, given a single SYT, numerically constructs the corresponding Bethe roots.

Rather than attacking the Bethe equations directly in root variables, the pipeline uses the modern *Q-system* reformulation of the nested Bethe ansatz, in which the fundamental objects are polynomials $Q_{a,s}(u)$ of the spectral parameter u , attached to nodes (a, s) of a lattice built from the Young diagram, and required to satisfy a discrete bilinear functional relation (Section 2). The distinguishing feature of this project is that the Q-system is not solved by treating the coefficients of every $Q_{a,s}$ as independent unknowns; instead, a closed-form solution is constructed directly as a **Wronskian (Casoratian) determinant** of a small number of elementary building blocks, split into a *bosonic* and a *fermionic* family according to a chosen *marked point* on the diagram (Section 3). This is the origin of the name (*SUSY*) *Wronskian Bethe Equations* carried by the project.

1.2 Numerical strategy

Even after the Wronskian ansatz reduces the problem to finitely many polynomial equations in finitely many coefficients, those equations must still be solved numerically for each SYT. The pipeline does this by introducing a single auxiliary parameter Λ — realized physically as one chain site’s inhomogeneity — and constructing, at $\Lambda \rightarrow \infty$, an exact combinatorial solution associated with the tableau’s shape (Section 4). The physical, homogeneous point $\Lambda = 0$ is then reached by *numerically continuing* this solution as Λ is driven from its large starting value to zero — a textbook parameter homotopy continuation problem, which we formalize in Section 6. The tableau itself is built up one box at a time, each new box’s site briefly appearing at large Λ and then being continued into the homogeneous chain, so that a single SYT computation is in fact a sequence of $|\lambda|$ homotopy continuations, each seeded by the previous one’s converged solution (Section 4.3).

The actual path tracking is delegated to **Bertini**, a numerical algebraic geometry package implementing adaptive multiple-precision predictor-corrector continuation, through its Python interface. The symbolic construction of the Wronskian relations, and the orchestration of many such continuations across all SYT of a Young diagram, is carried out in the Wolfram Language; the two layers communicate through an explicit polynomial system exported to a plain-text file. We describe this numerical layer formally in Section 6 and summarize its software realization in Section 8.

1.3 Scope and evidentiary status of this report

This report is based on a direct reading of the project’s source files. A number of internal comments cite specific equation numbers of an external reference (“Eq. 2.3”, “Eq. 2.33”, “Eq. 5.7”, “Eq. 5.8”), indicating that the construction of Section 3 is a direct numerical implementation of a specific paper; that paper is not included in the repository. Where this report states a physical interpretation that

is not explicit in an in-code comment, this is flagged as an inference from the code’s structure rather than a fact quoted from a reference. The mathematical structure itself — the Q-system, the Wronskian solution, the marked-point/grading freedom, the vacancy-number combinatorics, and the numerical continuation scheme — is, by contrast, directly and unambiguously present in the source and is derived formally below.

1.4 Outline

Section 2 introduces the Q-system and explains the Wronskian mechanism behind the Hirota/QQ relation. Section 3 constructs the supersymmetric Wronskian solution and its marked-point freedom. Section 4 introduces the deformation parameter Λ , its asymptotic solution, and the sequential construction. Section 5 develops the rigged-configuration / string-hypothesis combinatorics and the flip symmetry used for validation. Section 6 formalizes homotopy continuation and Bertini’s realization of it. Section 7 works every preceding definition out by hand on the smallest nontrivial shape. Section 8 summarizes the software pipeline. Section 9 reports validation and performance results. Section 10 discusses scope, limitations, and outlook. Table 1 below collects the recurring notation for reference.

Symbol	Meaning
$\lambda \vdash L, \lambda'$	Young diagram (L boxes) labeling a multiplet, and its conjugate
SYT	standard Young tableau: a filling of λ labeling one eigenstate
u	spectral parameter (rapidity)
$Q_{a,s}(u)$	Q-function at lattice node (a, s) ; $M(a, s)$ its polynomial degree
$\theta_1, \dots, \theta_L$	chain-site inhomogeneities; all zero for a homogeneous chain
$\text{Wr}[f, g]$	discrete Wronskian (Casoratian) of f, g , Eq. (5)
$\text{mP} = (\lambda_*, \nu_*)$	marked point: bosonic/fermionic split of elementary Q-functions
$Q_{A J}, \gamma_{A,J,k}$	Q-function (resp. its coefficients) for bosonic set A , fermionic set J
$c_{a,s,k}$	generic-basis Taylor coefficients of $Q_{a,s}$ (Section 3.3)
Λ, Λ_0	continuation (deformation) parameter and its large starting value
$n_s^{(a)}, J, P_s^{(a)}$	string multiplicity, rigging, and vacancy number (Section 5)
$t, H(x, t)$	homotopy parameter and homotopy function (Section 6)

Table 1: Recurring notation.

2 The Q-System Formalism

Written directly in terms of Bethe roots, the equations of a nested Bethe ansatz are a system of coupled transcendental equations in which every root of every family scatters off every other root of every family: even before any numerics, simply *writing down* the equations for a chain with several nesting levels and more than a handful of roots per level is already unwieldy, and finding a good starting guess for a numerical solver is far from obvious. The Q-system reformulation sidesteps both difficulties at once. Rather than working with the roots themselves, one works with the (much better-behaved) generating polynomials that have those roots as their zeros; the coupling between different families of roots is then encoded not in a system of rational equations but in a single, uniform bilinear *functional* relation between neighboring polynomials, Eq. (3) below, which is the same at every node of the lattice regardless of how many roots are involved. This uniformity is exactly what makes the construction amenable to the closed-form (Wronskian) solution method of

the rest of this section, and to the systematic numerical continuation of Section 6.

2.1 Q-functions on the Young diagram

Let $\lambda = (\lambda_1 \geq \lambda_2 \geq \dots)$ be a partition of L , drawn as a Young diagram, and let λ' denote its conjugate partition. To each lattice point (a, s) with $0 \leq a, 0 \leq s$ we associate a monic polynomial $Q_{a,s}(u)$ of the spectral parameter u , of degree

$$M(a, s) = L + a s - \sum_{i=1}^a \lambda_i - \sum_{j=1}^s \lambda'_j, \quad (1)$$

so that $M(a, s) = 0$ once (a, s) leaves the diagram's "co-hook" (the region where the formula would otherwise become negative); such nodes carry the trivial function $Q_{a,s} \equiv 1$. The distinguished node $(0, 0)$ has $M(0, 0) = L$ and its polynomial,

$$Q_{0,0}(u) = \prod_{n=1}^L (u - \theta_n), \quad (2)$$

is fixed entirely by the chain's L inhomogeneities $\theta_1, \dots, \theta_L$ (one per site); for a homogeneous chain, all $\theta_n = 0$ except possibly a single deformed site, as in Section 4.

Example 2.1 (A running example: $\lambda = (2, 1)$). *Throughout this report we will repeatedly return to the smallest Young diagram with more than one row, $\lambda = (2, 1)$: two rows, of lengths 2 and 1, three boxes in total ($L = 3$). Its conjugate is $\lambda' = (2, 1)$ as well (the diagram is self-conjugate: two boxes stacked in the first column, one box in the second). This shape has exactly two standard fillings,*

$$T_A = \begin{smallmatrix} 1 & 2 \\ 3 \end{smallmatrix}, \quad T_B = \begin{smallmatrix} 1 & 3 \\ 2 \end{smallmatrix},$$

matching the hook-length count $3!/(3 \cdot 1 \cdot 1) = 2$; in the language of the symmetric group S_3 acting on three objects, $\lambda = (2, 1)$ is the two-dimensional "standard" irreducible representation, and T_A, T_B label a basis of it. Evaluating the degree formula (1) at every node with $a + s \leq 2$ gives

$$M(0, 0) = 3, \quad M(1, 0) = M(0, 1) = 3 - 2 = 1, \quad M(1, 1) = M(2, 0) = M(0, 2) = 0,$$

so only three nodes carry a nontrivial polynomial: the length-3 vacuum $Q_{0,0}$, and two length-1 (linear) polynomials $Q_{1,0}, Q_{0,1}$ — each with a single root. Since λ has two rows, there is exactly one nesting level (Eq. (17) below), so this shape's Bethe ansatz has, per eigenstate, a single Bethe root: the zero of $Q_{1,0}$.

2.2 The Hirota (QQ) equation

Write $f^{[\pm k]}(u) := f(u \pm \frac{ik}{2})$ for a shift of the spectral parameter by k half-units along the imaginary axis. The Q-functions of the nested Bethe ansatz are required to satisfy, at every node (a, s) for which all four neighbors are defined,

$$Q_{a,s}^{[+1]} Q_{a+1,s+1}^{[-1]} - Q_{a,s}^{[-1]} Q_{a+1,s+1}^{[+1]} = i(M(a, s) - M(a + 1, s + 1)) Q_{a+1,s}(u) Q_{a,s+1}(u). \quad (3)$$

This is the (additive-normalization) *Hirota bilinear equation*, also called the *QQ-relation*, of the discrete integrable hierarchy underlying the nested Bethe ansatz. It is, on the face of it, a transcendental functional equation; the project's numerical strategy depends on converting it into finitely many *polynomial* equations in finitely many unknowns, which is accomplished in two complementary ways described below.

Polynomial form

If every $Q_{a,s}$ appearing in (3) is taken to be a polynomial of the degree prescribed by (1), with undetermined Taylor coefficients $c_{a,s,k}$,

$$Q_{a,s}(u) = u^{M(a,s)} + \sum_{k=1}^{M(a,s)} c_{a,s,k} u^{M(a,s)-k}, \quad (4)$$

then the left- and right-hand sides of (3) are themselves polynomials in u (the half-integer shifts only ever produce integer powers of i once the two half-shifted products are subtracted), and matching coefficients of each power of u turns (3) into a finite list of *polynomial equations* in the $c_{a,s,k}$. This “generic” route is available in the codebase and defines the ambient polynomial system that any solution — however constructed — must satisfy.

Wronskian (Casoratian) solution

The project’s principal method does not solve the polynomial system of the previous paragraph directly. Instead it *generates* a family of functions satisfying (3) identically, by representing each $Q_{a,s}$ as a determinant of shifted copies of a common set of “elementary” functions — a discrete analogue of the Wronskian determinant of solutions of a linear ODE, sometimes called a *Casoratian*. Concretely, define the discrete Wronskian of two functions f, g by

$$\text{Wr}[f, g](u) := f^{[+1]}(u) g^{[-1]}(u) - f^{[-1]}(u) g^{[+1]}(u). \quad (5)$$

Why should a determinant of this kind have any chance of satisfying a bilinear relation like (3)? The intuition is a discrete analogue of a classical fact about ordinary Wronskians: if two functions f, g both solve a common linear second-order difference (“Baxter-type”) equation with shared potential $V(u)$,

$$X(u+i) + X(u-i) = V(u) X(u), \quad X \in \{f, g\}, \quad (6)$$

then their discrete Wronskian is a *conserved quantity* of the shift:

$$\begin{aligned} \text{Wr}[f, g]^{[+1]}(u) - \text{Wr}[f, g]^{[-1]}(u) &= g(u)[f(u+i)+f(u-i)] - f(u)[g(u+i)+g(u-i)] \\ &= V(u)[g(u)f(u) - f(u)g(u)] = 0, \end{aligned} \quad (7)$$

by direct substitution of (6) for both f and g — exactly the discrete counterpart of the elementary fact that the Wronskian of two solutions of a linear ODE with no first-derivative term is constant. This is the generic phenomenon exploited throughout the Q-system literature: elementary Q-functions are required to be solutions of a shared linear difference equation fixed by the vacuum data (2), and Casoratians of such solutions automatically satisfy conservation-law-type bilinear identities under the shift $u \rightarrow u \pm i$.

Remark 2.1 (What is, and is not, claimed here). *Equation (7) is only the simplest instance of this mechanism (a vanishing right-hand side, for two solutions of the same, ungraded linear equation). The actual QQ-relation (3) that the project’s Q-system must satisfy has a nonzero right-hand side $i(M_{a,s} - M_{a+1,s+1}) Q_{a+1,s} Q_{a,s+1}$, and its bosonic and fermionic elementary building blocks (Section 3) are solutions of two different, graded linear problems, not one shared ungraded equation. Establishing that the specific super-Wronskian ansatz used here satisfies every instance of (3) on the full lattice — not merely the toy identity (7) — is a genuine consistency theorem about that graded construction; it belongs to the reference work the implementation follows (cited there by equation*

number, as noted in the Introduction) and is not re-derived from first principles in this report. What is directly confirmed in the source code, and is recorded below exactly as implemented, is the recursive definition by which every higher Q -function is generated from elementary ones once that consistency is assumed.

The project’s implementation makes this recursion explicit and constructive: writing $Q_{A|J}$ for the Q -function attached to a bosonic index set A and a fermionic index set J (Section 3), one has the reduction

$$Q_{A_0 \cup \{a,b\} | J} = \frac{\text{Wr}[Q_{A_0 \cup \{a\} | J}, Q_{A_0 \cup \{b\} | J}]}{Q_{A_0 | J}}, \quad (8)$$

$$Q_{A | J_0 \cup \{i,j\}} = \frac{\text{Wr}[Q_{A | J_0 \cup \{i\}}, Q_{A | J_0 \cup \{j\}}]}{Q_{A | J_0}}, \quad (9)$$

i.e. a Q -function with one more bosonic (resp. fermionic) index than $Q_{A_0 | J}$ is the ratio of the Wronskian of its two “parents” to $Q_{A_0 | J}$ itself — the discrete analogue of a classical Plücker relation among Wronskians of nested subsets of solutions. Every multi-index Q -function of the system is generated from single-index ($|A| = |J| = 1$, or one of A, J empty) elementary Q -functions by iterating (8)–(9).

3 The Supersymmetric Wronskian and the Marked-Point Freedom

3.1 Marked points and gradings

The recursion (8)–(9) treats the elementary building blocks as falling into two families, indexed by disjoint label sets: a *bosonic* family of size λ_* and a *fermionic* family of size ν_* , with $\lambda_* + \nu_*$ equal to the number of rows of the Young diagram participating in the construction. The project calls the choice $\text{mP} = (\lambda_*, \nu_*)$ a **marked point**: it fixes how many of the diagram’s rows are assigned to the bosonic family versus the fermionic one. In the representation theory of a superalgebra $\mathfrak{gl}(\lambda_* | \nu_*)$, this is precisely the datum of a choice of *simple-root system* (a **Kac–Dynkin diagram**): unlike an ordinary Lie algebra, a Lie superalgebra admits several inequivalent but representation-theoretically equivalent Dynkin diagrams, related by “fermionic duality” moves, and different choices lead to Wronskian solutions of different total polynomial degree. The implementation makes this freedom explicit by enumerating admissible marked points/paths across the diagram and selecting one that minimizes the total number of Bethe roots to be computed — a computational-efficiency criterion, not a physical one, applied on top of the mathematical freedom just described.

Remark 3.1. *The repository does not state explicitly whether the physical spin chain being solved is itself supersymmetric, or whether the bosonic/fermionic split is used purely as a computational device that produces an efficient closed-form Wronskian solution for an ordinary (non-super) nested Bethe ansatz problem. Both readings are consistent with the mathematics above; this report presents the construction in a manner agnostic to that distinction and defers to the external source cited internally by equation number for the precise physical statement.*

Example 3.1 (Marked points for $\lambda = (2, 1)$). *Continuing Example 2.1: the diagram has two rows, so a marked point is a splitting $2 = \lambda_* + \nu_*$, and there are three choices, $(\lambda_*, \nu_*) \in \{(2, 0), (1, 1), (0, 2)\}$. The choice $(2, 0)$ assigns both rows to the bosonic family and none to the fermionic one: the super-Cauchy determinant (13) below degenerates to an ordinary 2×2 Wronskian determinant of two purely bosonic towers, with no fermionic block at all — the “purely bosonic” grading. The choice $(1, 1)$ instead assigns one row to each family, mixing a single bosonic and a single fermionic elementary Q -function; the choice $(0, 2)$ assigns both rows to the fermionic family. All three choices*

describe the same vacuum polynomial $Q_{0,0}$, by construction, once each is separately fit to the target inhomogeneities (14) — they differ only in how many elementary coefficients must be solved for along the way, which is exactly the efficiency trade-off the marked-point search of Section 8 is built to exploit.

3.2 The super-Wronskian determinant

Fix a marked point $\text{mP} = (\lambda_*, \nu_*)$. Denote the λ_* bosonic elementary Q-functions by $Q_{\{p\}|\emptyset}$, $p = 1, \dots, \lambda_*$, and the ν_* fermionic ones by $Q_{\emptyset|\{q\}}$, $q = 1, \dots, \nu_*$. Form the two rectangular blocks of shifted elementary functions

$$B_{p,j}(u) = Q_{\{p\}|\emptyset}\left(u + \frac{i}{2}(\lambda_* - \nu_* - 2j + 1)\right), \quad 1 \leq p \leq \lambda_*, \quad 1 \leq j \leq \lambda_* - \nu_*, \quad (10)$$

$$F_{p,q}(u) = Q_{\{p\}|\{q\}}\left(u + \frac{i}{2}(\lambda_* - \nu_*)\right), \quad 1 \leq p \leq \lambda_*, \quad 1 \leq q \leq \nu_*, \quad (11)$$

where the mixed bosonic–fermionic building block $Q_{\{p\}|\{q\}}$ is itself fixed, for each single pair (p, q) , by demanding that it solve the elementary instance of (3) coupling the purely bosonic and purely fermionic elementary Q-functions,

$$Q_{\{p\}|\{q\}}^{[+1]} - Q_{\{p\}|\{q\}}^{[-1]} = Q_{\{p\}|\emptyset} Q_{\emptyset|\{q\}}, \quad (12)$$

solved order by order in u for the coefficients of $Q_{\{p\}|\{q\}}$ (this is a linear recursion in the Taylor coefficients, since the right-hand side is already known once the bosonic and fermionic elementary blocks are fixed). The vacuum Q-function is then the single $\lambda_* \times \lambda_*$ super-Cauchy/Wronskian determinant

$$Q_{0,0}(u) = (-1)^{\nu_*(\lambda_* - \nu_*)} \det[F \mid B](u), \quad (13)$$

i.e. the determinant of the $\lambda_* \times \lambda_*$ matrix whose first ν_* columns are the fermionic block F and whose remaining $\lambda_* - \nu_*$ columns are the bosonic block B . Built up by iterating the Wronskian-quotient recursion (8)–(9) from the elementary bosonic and fermionic blocks, (13) automatically satisfies the QQ-relation (3) against its neighbors for *any* choice of the elementary block coefficients (Remark following Eq. (7)); the only remaining task is to fix those coefficients so that (13) reproduces the physical vacuum polynomial (2), i.e. so that

$$\text{monic}[Q_{0,0}^{\text{Wronskian}}(u)] = \prod_{n=1}^L (u - \theta_n), \quad (14)$$

which, after clearing the overall normalization and matching coefficients of each power of u , is a finite polynomial system in the elementary blocks' coefficients — the **Wronskian relations** that are ultimately handed to the numerical continuation of Section 6.

3.3 Two coefficient bases

Because the physical content of the Q-system is carried by the elementary bosonic/fermionic building blocks, while the generic polynomial system of Section 2.2 is expressed in the coefficients $c_{a,s,k}$ of *every* node, the implementation maintains an explicit two-way translation between:

- the **generic basis**, $\{c_{a,s,k}\}$, one set of Taylor coefficients per Young-diagram node;
- the **Wronskian basis**, $\{\gamma_{A,J,k}\}$, the Taylor coefficients of the elementary bosonic/fermionic blocks introduced above.

The translation is carried out by a *triangular elimination*: at each stage, one selects a coefficient-matching equation from (14) (or its analogue for other nodes) that contains exactly one still-undetermined variable while every other symbol appearing in it is already known, and solves for that variable; the process is iterated until either every unknown has been eliminated or no such equation remains (in which case the system is under-determined in that basis, a known limitation for certain small diagrams). This is the formal content of what the implementation calls, informally, the “strict triangular sort”; only the Wronskian-basis coefficients $\gamma_{A,J,k}$ are exported as unknowns to the numerical continuation step, and the generic basis is recovered afterward, purely as a convenience for extracting Bethe roots (Section 4.4 below).

4 The Deformation Parameter and the Sequential Construction

4.1 The chain inhomogeneity as a homotopy parameter

Consider building the tableau’s chain one site at a time. After the $(n-1)$ -th site has already been solved (all its Wronskian-basis coefficients fixed at the physical, homogeneous point), the n -th site is introduced by setting its inhomogeneity to a formal parameter Λ while every earlier site retains its already-converged, homogeneous value of 0:

$$Q_{0,0}^{(n)}(u; \Lambda) = u^{n-1} (u - \Lambda). \quad (15)$$

The physical (homogeneous) chain of length n is recovered at $\Lambda = 0$; the value $\Lambda \rightarrow \infty$ instead decouples the new site entirely, in the following precise sense.

4.2 Asymptotic solution at large Λ

Every root of every Q-function in the Wronskian construction can be tracked, order by order in $1/\Lambda$, by attaching to *each root* a formal weight built from the tableau’s combinatorics: writing θ_m for a single formal deformation parameter attached to tableau entry $m \in \{1, \dots, L\}$, one sets

$$q_{a,s,k} = \left(\prod \text{hook-content ratios of the sub-tableau at } (a, s, k) \right) \theta_{m(a,s,k)}, \quad (16)$$

a combinatorial (rational-function-of-diagram-data) weight multiplying the single formal parameter of whichever tableau entry that root is associated with. Specializing $\theta_m = \Lambda$ for the newest entry ($m = n$) and $\theta_m = 0$ for every earlier entry, and substituting into the Wronskian relations (14) together with (15), yields a system that is exactly solvable — typically linear or of very low degree — because the bulk of the tableau’s roots are pinned to $\theta_m = 0$ and only the newest entry’s roots carry any Λ -dependence at all. This closed-form (or near-closed-form) solution, valid asymptotically as $\Lambda \rightarrow \infty$, supplies the starting point for the numerical continuation: no root-finding is required to seed it, only the combinatorial evaluation of (16) on the given tableau.

4.3 Sequential construction

The full computation for a target SYT of shape $\lambda \vdash L$ proceeds inductively on the chain of sub-tableaux obtained by restricting the target tableau to entries $\{1\}, \{1, 2\}, \dots, \{1, \dots, L\}$ (the tableau’s own growth sequence, a saturated chain $\emptyset = \lambda^{(0)} \subset \lambda^{(1)} \subset \dots \subset \lambda^{(L)} = \lambda$ in Young’s lattice):

1. at $n = 1$, the one-site chain is solved by the exact asymptotic construction of the previous subsection, continued (trivially) to $\Lambda = 0$;

2. at each subsequent n , the $(n-1)$ -site converged solution is held fixed, the new site's inhomogeneity is set to a symbolic Λ as in (15), the exact large- Λ solution (16) is evaluated as a numerical starting point at a large finite value of Λ , and the Wronskian relations are then numerically continued in Λ from that large value down to $\Lambda = 0$ (Section 6), yielding the converged n -site solution.

Different standard Young tableaux of the same shape correspond to different orderings of this box-by-box growth, hence to different continuation paths; they are expected, on general representation-theoretic grounds, to terminate on the distinct members of the degenerate eigenspace associated with that shape. The total computation for one SYT is therefore not a single homotopy continuation but a sequence of $L - 1$ of them (the first site requiring none), each a full instance of the formalism developed in Section 6, seeded by the previous one's output.

Example 4.1 (Growth sequences for T_A and T_B). *Continuing Example 2.1, the two tableaux of shape $(2, 1)$ trace out different intermediate shapes on their way to $\lambda = (2, 1)$, even though they end at the same shape:*

$$T_A = \frac{1}{3}^2 : \quad \emptyset \rightarrow (1) \rightarrow (2) \rightarrow (2, 1), \quad T_B = \frac{1}{2}^3 : \quad \emptyset \rightarrow (1) \rightarrow (1, 1) \rightarrow (2, 1).$$

Both start identically (a single site, trivially continued) and end identically (three sites, shape $(2, 1)$), but their second step continues from a one-site chain to a genuinely different two-site sub-problem: T_A 's second site is added to the first row (an intermediate shape (2) , with no second row yet, hence a trivial, un-nested Q -system at that step), while T_B 's second site opens the second row (an intermediate shape $(1, 1)$, already carrying one nesting level). This concretely illustrates why the two tableaux, despite sharing a final shape and hence a final set of Wronskian relations (14), are continued along different paths and are expected to converge on the two distinct eigenstates that shape $(2, 1)$'s degenerate multiplet contains.

4.4 Extraction of Bethe roots

Once the coefficients of every node have been recovered in the generic basis $\{c_{a,s,k}\}$ (Section 3.3) at the physical point $\Lambda = 0$ for the full tableau, the Bethe roots themselves are the zeros of the *first-column* Q -functions,

$$Q_{w,0}(u) = 0, \quad w = 1, \dots, (\text{number of rows of } \lambda) - 1, \quad (17)$$

one independent root set per nesting level w , consistent with a nested Bethe ansatz of rank equal to the number of rows of the diagram.

5 Rigged Configurations, the String Hypothesis, and a Flip Symmetry

5.1 Strings and vacancy numbers

Independently of the Wronskian construction above, the classical *string hypothesis* conjectures that solutions of the (ordinary, non-super) nested Bethe equations organize into **strings**: for each node a of the auxiliary root system, a collection of complex roots sharing a common real center v and equally spaced along the imaginary direction,

$$u_k = v - \frac{i}{2}(s - 1 - 2k), \quad k = 0, 1, \dots, s - 1, \quad (18)$$

for a string of length s . A configuration of strings is encoded combinatorially by a **rigged configuration**: for each node a and each string length s , a nonnegative integer multiplicity $n_s^{(a)}$ together with, for every one of those $n_s^{(a)}$ strings, an integer **rigging** $J \in \{0, 1, \dots, P_s^{(a)}\}$ bounded by the node's **vacancy number**

$$P_s^{(a)} = \delta_{a,1} L - \sum_b A_{ab} \sum_{s'} \min(s, s') n_{s'}^{(b)}, \quad (19)$$

where A_{ab} is the Cartan matrix of the auxiliary (type- A_{n-1} , in the cases implemented here) root system. Equation (19) is the standard counting-function/vacancy formula of the string hypothesis: $\delta_{a,1} L$ is the leading contribution of the physical chain to the first node, and the Cartan-matrix contraction against the “string-string scattering kernel” $\min(s, s')$ subtracts, for every other string present at neighboring nodes, the number of momentum slots it excludes at node a . A rigging J for a given string is then, in the usual interpretation, a choice of one of the $P_s^{(a)} + 1$ admissible mode numbers left available after this exclusion — exactly analogous to filling single-particle levels subject to a generalized Pauli principle. The number of admissible riggings for a fixed string-length content is therefore

$$\prod_{a,s} \binom{P_s^{(a)} + n_s^{(a)}}{n_s^{(a)}}, \quad (20)$$

a product of binomial coefficients that (summed appropriately over all admissible string contents of a given node) reproduces the Kostka-number counting of standard Young tableaux — the combinatorial statement of completeness of the Bethe ansatz.

Example 5.1 (Vacancy numbers for $\lambda = (2, 1)$). *The shape $(2, 1)$ of Example 2.1 has a single nesting level (Eq. (17)), so its auxiliary root system has a single node $a = 1$, with Cartan entry $A_{11} = 2$. Consider the string content with one string of length $s = 1$ and nothing else, $n_1^{(1)} = 1$: by (19),*

$$P_1^{(1)} = \delta_{1,1} \cdot 3 - A_{11} \cdot \min(1, 1) \cdot n_1^{(1)} = 3 - 2 \cdot 1 \cdot 1 = 1,$$

so the single string's rigging can be $J = 0$ or $J = 1$: exactly $\binom{P_1^{(1)} + n_1^{(1)}}{n_1^{(1)}} = \binom{1+1}{1} = 2$ admissible riggings, matching the two standard Young tableaux T_A, T_B of that shape one for one. This is the smallest nontrivial instance of the general fact (20): the rigged-configuration count of admissible riggings for a fixed string content reproduces the number of standard Young tableaux of the corresponding shape.

5.2 The Kerov–Kirillov–Reshetikhin bijection

The **Kerov–Kirillov–Reshetikhin (KKR) bijection** is a classical, constructive correspondence between rigged configurations and standard Young tableaux (equivalently, crystal paths in a tensor product of single-box representations). The project implements both directions of this bijection explicitly, following the classical algorithm: recursively remove (or add) the box associated with the current largest (or next) tableau entry, choosing at each step the unique *singular* string — one whose rigging equals its vacancy number exactly — as the string a box is added to or removed from, and updating every vacancy number via (19) after each such move. The construction converts a rigged configuration into explicit **mode numbers** for each string (via a further shift of the rigging by the vacancy number and the string index) which, through the string ansatz (18), give an explicit approximate (string-hypothesis) prediction for the Bethe roots associated with a tableau, independent of the exact Wronskian construction of Sections 2–3.

5.3 A flip involution and a conjectured root symmetry

Complementing every string’s rigging against its own vacancy number,

$$J \longmapsto P_s^{(a)} - J, \quad (21)$$

applied simultaneously to every string of a rigged configuration, defines an involution on rigged configurations (and, via the KKR bijection, an involution on standard Young tableaux of a fixed shape), which we denote $\text{SYT} \mapsto \text{Flip}(\text{SYT})$.

Empirical Symmetry 5.1 (Root negation under the flip). *For a standard Young tableau SYT with Bethe roots $\{u_1, \dots, u_r\}$ (at any fixed nesting level), the flipped tableau’s Bethe roots satisfy*

$$\{u_1, \dots, u_r\} = \{-u_1, \dots, -u_r\} \quad \text{as computed for } \text{Flip}(\text{SYT}). \quad (22)$$

This statement is not derived in the codebase from first principles; it is used purely as an *empirical, numerically checked* consistency condition (Section 9.2). Combinatorially, the flip (21) coincides, on standard Young tableaux, with **Schützenberger evacuation** — the classical involution obtained by repeatedly removing the box containing the tableau’s smallest entry via jeu-de-taquin slides and relabeling — a fact from crystal-base theory (evacuation realizes the crystal-graph analogue of complex conjugation of the underlying representation). The project exploits this coincidence purely as a fast, dependency-free way to compute the flipped tableau without constructing its rigged configuration explicitly, and treats Conjecture 5.1 as an independent numerical cross-check on the Wronskian-continuation output of Sections 2–4 rather than as a derivation of it.

6 Numerical Continuation: Formalism and Realization

6.1 A concrete warm-up example

Before setting up the general formalism, it is worth fixing ideas with the simplest possible instance, reproduced directly from the project’s own introductory material. Take the target system

$$F(x, y) = \begin{pmatrix} x^2 + y^2 - 1 \\ x + y \end{pmatrix} = 0, \quad (23)$$

i.e. the intersection of the unit circle with the line $y = -x$, with exact solutions $(x, y) = (\pm \frac{1}{\sqrt{2}}, \mp \frac{1}{\sqrt{2}})$. This system has no free parameter to continue in, so one instead introduces an artificial *start system* $G(x, y) = 0$ of the same degrees (2 and 1, respectively) with known solutions — a **total-degree start system** — and a homotopy $H = (1 - t)F + tG$ exactly as in (24) below. Bézout’s theorem bounds the number of solution paths that must be tracked by the product of the two degrees, $2 \times 1 = 2$; correspondingly there are two start points to track, one per path, from $t = 1$ down to $t = 0$. Tracking both paths with an adaptive multiple-precision corrector (as described below) recovers the two exact intersection points to the tracker’s tolerance, with real parts $\approx \pm 0.70710678\dots = \pm 1/\sqrt{2}$ — this is exactly what one finds running Bertini on (23), and it is the same mechanism, applied to a genuine polynomial system with no natural deformation parameter of its own, that Section 6.3 specializes to the project’s actual problem, where instead of an artificial start system one already has an *exact* solution at $\Lambda = \Lambda_0$ to continue from.

6.2 Homotopy continuation and the Davidenko equation

Let $F : \mathbb{C}^N \rightarrow \mathbb{C}^N$ be the target polynomial system (here, the Wronskian relations (14) at $\Lambda = 0$, in the coefficients γ of Section 3.3), and let G be a *start system* with known solution(s), continuously

connected to F by a homotopy

$$H(x, t) = (1 - t)F(x) + tG(x), \quad t \in [0, 1], \quad (24)$$

so that $H(x, 1) = G(x)$ and $H(x, 0) = F(x)$. Assuming the solution path $x(t)$ defined implicitly by $H(x(t), t) = 0$ is smooth, differentiating with respect to t gives the **Dauidenko differential equation**,

$$J_x H(x(t), t) \dot{x}(t) + \partial_t H(x(t), t) = 0 \implies \dot{x}(t) = -[J_x H(x(t), t)]^{-1} \partial_t H(x(t), t), \quad (25)$$

an ordinary differential equation for the path $x(t)$, provided the Jacobian $J_x H$ remains nonsingular along the path. Numerically tracking the path from $t = 1$ to $t = 0$ then reduces to (i) a **predictor** step that numerically integrates (25) by one of the standard explicit ODE schemes, and (ii) a **corrector** step that removes the resulting drift off the true solution path by Newton iteration,

$$x_{k+1} = x_k - [J_x H(x_k, t)]^{-1} H(x_k, t), \quad (26)$$

applied at fixed t until convergence to a specified tolerance. This predictor–corrector scheme, together with *adaptive* control of both the step size in t and the working numerical precision (raised whenever the Jacobian in (26) becomes ill-conditioned, as happens near a path singularity), is exactly what Bertini’s **adaptive multiple-precision tracker** (“AMPTracker”) implements; the several named predictors available (constant, Euler, Heun, and a family of embedded Runge–Kutta schemes of increasing order) are simply different choices of explicit ODE integrator for (25), trading predictor accuracy (and hence achievable step size) against per-step cost.

6.3 Specialization: a linear parameter homotopy in Λ

The construction of Section 4 is a particularly favorable instance of (24): rather than requiring an artificial start system, the Wronskian relations (14), left symbolic in Λ , already constitute a one-parameter polynomial family $F_\Lambda(\gamma) = 0$ whose member at a large value $\Lambda = \Lambda_0$ has the exact combinatorial solution of Section 4, and whose member at $\Lambda = 0$ is the physical system. Introducing the affine reparametrization

$$\Lambda(t) = (1 - t) \cdot 0 + t \cdot \Lambda_0 = t \Lambda_0, \quad t \in [0, 1], \quad (27)$$

gives $H(\gamma, t) := F_{\Lambda(t)}(\gamma)$, with $H(\gamma, 1) = F_{\Lambda_0}(\gamma)$ solved exactly and $H(\gamma, 0) = F_0(\gamma)$ the target; exactly one path is tracked (there is no Bézout-number multiplicity of start points to enumerate, since the start point is already known exactly), from $t = 1$ to $t = 0$. This is the parameter homotopy actually solved by the numerical back end: the polynomial system exported at each step of the sequential construction (Section 4.3) is $F_\Lambda(\gamma)$ with Λ left as the single homotopy variable and (27) substituted in symbolically before the system is handed to the tracker, together with the exact starting value $\gamma(\Lambda_0)$ from Section 4 as the $t = 1$ start point.

6.4 Numerical safeguards

Two safeguards are imposed around the tracked endpoint before it is accepted as a physical solution. First, since the physical Bethe roots recovered downstream (Eq. (17)) are real linear combinations of the tracked coefficients for the tableaux considered here, each returned coefficient $\gamma = \gamma_{\text{re}} + i\gamma_{\text{im}}$ is required to satisfy

$$|\gamma_{\text{im}}| \leq \varepsilon_{\text{track}} \cdot \max(1, |\gamma_{\text{re}}|), \quad (28)$$

for the tracker’s own tolerance $\varepsilon_{\text{track}}$, with any violation (or any non-finite value) aborting the run rather than silently truncating an unexpectedly complex endpoint. Second, every numerical result is published to disk only by an atomic replace of a completed temporary file, so that a reader can never observe a partially written solution, and a failed or timed-out continuation simply leaves no new output rather than a corrupted one.

6.5 A path-tracking failure mode: branch jumps

The predictor–corrector scheme of (25)–(26) is only guaranteed to follow the correct solution branch if the step size in t is small enough, relative to the local curvature of the path and the separation of nearby solution branches, for the corrector to converge back to the *same* branch the predictor was following. When this fails — typically because the path passes close to another solution branch or to a singularity of the discriminant locus — the tracker can converge to a numerically valid but physically incorrect solution: a *branch jump*. Empirically, for a pair of tableaux related by the flip involution of Section 5.3 (whose roots are conjectured to be exact negatives of one another, Conjecture 5.1), starting the continuation from a substantially larger Λ_0 and tracking at higher precision and tighter tolerance was found to eliminate an observed branch-jump discrepancy entirely (Section 9.2); both adjustments reduce, respectively, the total path length that must be tracked and the size of the correction the Newton step must make at each point along it, and so plausibly reduce the risk of such a jump, though neither is a formal guarantee against it in general.

Remark 6.1 (Why extra precision helps). *It is worth making the mechanism a little more concrete. Two solution branches that pass close to one another correspond to the Jacobian $J_x H(x(t), t)$ in (26) becoming nearly singular somewhere along the path (formally, the path approaches the discriminant locus where two roots of the system coincide). Near such a point, the Newton correction $[J_x H]^{-1} H$ is the product of a huge, ill-conditioned inverse and a small residual, so the correction’s accuracy is extremely sensitive to round-off in H and $J_x H$ themselves: at fixed (say, machine) precision, the correction step can easily be thrown to the wrong one of the two nearby branches. Raising the working precision shrinks the round-off in H and $J_x H$ directly, which is exactly what lets the corrector resolve which of the two nearby branches it started closest to — this is the sense in which adaptive precision (raised locally, only where the path actually passes near such a point, rather than uniformly everywhere) is doing real, not merely cosmetic, numerical work.*

7 A Fully Worked Example: the Shape $(2, 1)$

It is worth pausing to see every piece introduced so far — the degree count, the number of nesting levels, the flip involution, and the expected root negation — meet in a single, completely explicit, hand-checkable calculation, before turning to the software that automates it for shapes too large to solve by hand.

For the purely bosonic marked point $(\lambda_*, \nu_*) = (2, 0)$ of Example 3.1, the shape $\lambda = (2, 1)$ reduces to the ordinary (non-super) nested Bethe ansatz for a homogeneous chain of $L = 3$ sites, with one nesting level and, per Example 2.1, a single Bethe root per eigenstate. This is precisely the textbook *one-magnon sector* of the periodic Heisenberg (XXX) spin chain: a state with a single magnon at rapidity u is an eigenstate of the transfer matrix provided the periodicity condition

$$\left(\frac{u + \frac{i}{2}}{u - \frac{i}{2}} \right)^L = 1 \quad (29)$$

holds — the $M = 1$ case of the Bethe equations, in which the usual scattering factors between magnons are simply absent. At $L = 3$, clearing denominators in (29) gives $(u + \frac{i}{2})^3 = (u - \frac{i}{2})^3$, and since $a^3 - b^3 = (a - b)(a^2 + ab + b^2)$ with $a - b = i \neq 0$, this reduces to the single quadratic

$$\left(u + \frac{i}{2}\right)^2 + \left(u + \frac{i}{2}\right)\left(u - \frac{i}{2}\right) + \left(u - \frac{i}{2}\right)^2 = 3u^2 - \frac{1}{4} = 0, \quad \implies \quad u = \pm \frac{1}{2\sqrt{3}}. \quad (30)$$

There are exactly two roots, matching the two standard Young tableaux T_A, T_B of Example 2.1 one for one: each tableau’s degree-one Q-function $Q_{1,0}(u) = u \mp \frac{1}{2\sqrt{3}}$ contributes one of the two roots of (30), consistent with the general multiplicity count of Section 5 (two admissible riggings, Example 5.1, for two SYT).

A direct jeu-de-taquin computation (the evacuation algorithm of Section 5.3) confirms that T_A and T_B are each other’s flip, $\text{Flip}(T_A) = T_B$: removing the box containing entry 1 from $T_A = \{\{1, 2\}, \{3\}\}$, sliding the resulting hole down and relabeling, reproduces $T_B = \{\{1, 3\}, \{2\}\}$ exactly. Conjecture 5.1 therefore predicts $\text{Roots}(T_A) = -\text{Roots}(T_B)$ — and indeed the two roots found in (30), $+\frac{1}{2\sqrt{3}}$ and $-\frac{1}{2\sqrt{3}}$, are exact negatives of one another. This is the smallest possible instance in which every structural claim of this report — the degree/multiplicity count, the two-tableaux labeling of a degenerate multiplet, and the flip-negation symmetry — can be checked by hand, with no numerical continuation required at all (the one-magnon equation (29) is exactly solvable). For any shape with more than one magnon per level, no closed form is available in general, and this is exactly where the asymptotic construction and homotopy continuation of Sections 4–6 take over.

8 Software Realization

This section summarizes, at the level of architecture rather than implementation detail, how the formalism of Sections 2–6 is realized as a working pipeline; it is included for completeness and to ground the validation results of Section 9, not as a substitute for the mathematical description above.

Symbolic layer (Wolfram Language). Given a Young diagram, a symbolic layer constructs the Q-system lattice, selects a marked point, builds the super-Wronskian expression (13), and derives the Wronskian relations (14) together with the two-basis translation of Section 3.3. A second, independent module implements the asymptotic construction of Section 4 and drives the sequential, box-by-box computation of Section 4.3 for a given standard Young tableau, delegating each individual continuation step to a pluggable numerical back end.

Numerical back end (Bertini, via Python). At each continuation step, the current Wronskian relations (with Λ left symbolic per (27)) and a high-precision starting point are exported to a plain-text table (one polynomial equation per row, each row carrying the variable name, its starting value, and its defining expression). A Python driver parses this table into a genuine polynomial system, forms the parameter path of (27), and tracks a single path with Bertini’s adaptive multiple-precision tracker (Section 6), exposing the tracker’s tolerance, precision, step-size, Newton-iteration, and predictor-scheme parameters as configurable settings. The converged coefficients are validated against the reality condition (28) and published atomically; the symbolic layer imports them and proceeds to the next site. *(An earlier version of this back end delegated the same step to a Julia-based homotopy-continuation package; several internal option and file names still reflect that earlier design, though the currently used back end is Bertini only.)*

Orchestration and batch execution. A thin orchestration layer isolates every numerical global used by the symbolic/asymptotic construction on a per-task basis, allocates a dedicated working

directory and log files per tableau, and records detailed per-stage timing. A shell-level batch launcher runs many tableaux (optionally all standard Young tableaux of a given shape) as independent, parallel operating-system processes, consolidating their individual results into a single aggregate table under a file lock, atomically, so that a batch’s partial results remain safely readable while it is still running; a native desktop application provides the same set of operations (single-tableau runs, batch runs, the flip cross-check of Section 9.2, and result visualization) through a graphical interface, without introducing any code path distinct from the command-line tools.

Visualization. Bethe roots are visualized either in a notebook (for interactive inspection and for the negated-overlay comparison used in the flip check) or by a small, dependency-free tool that renders each nesting level of (17) as a panel of a hand-written vector-graphics document, automatically switching to a density-aggregated rendering when the number of roots in a batch result is too large to plot individually.

Remark 8.1 (Why split the pipeline this way). *The division between a symbolic layer and a numerical back end mirrors the division between Sections 2–4 (which never involve floating-point arithmetic — everything is exact combinatorics and exact or symbolic algebra) and Section 6 (the one place genuine numerical approximation enters). Consequently, the only interface the two layers need is the plain-text polynomial table described above: the symbolic layer never needs to know how a step’s solution was obtained, only that it converged, which is precisely what allowed the back end to be replaced (Section 8’s remark on the earlier Julia back end) without touching the mathematics of Sections 2–4 at all.*

9 Validation and Numerical Results

9.1 Regression testing

The numeric-string handling between Bertini’s long-decimal output and the symbolic layer’s arbitrary-precision numbers is directly tested: a 100-digit decimal value is required to round-trip through export and import with every digit intact, standard and scientific notations are required to parse correctly, and malformed or non-finite input is required to be rejected rather than silently coerced. Separately, a suite of saved reference tableaux is rerun from scratch and compared against their saved solutions *level by level* (per Eq. (17), never flattened across nesting levels), using an exhaustive one-to-one matching over root permutations at a default tolerance of 10^{-8} .

9.2 Validation of Conjecture 5.1: the flip check

The empirical symmetry of Conjecture 5.1 is checked numerically by computing, for a tableau and its flip, the **symmetric Hausdorff distance** between one root set and the negation of the other,

$$d_H(A, B) = \max \left\{ \max_{a \in A} \min_{b \in B} |a - b|, \max_{b \in B} \min_{a \in A} |a - b| \right\}, \quad B = -\text{Roots}[\text{Flip}(\text{SYT})], \quad (31)$$

computed at high (100-digit) arithmetic precision, with a default pass tolerance of 10^{-5} ; the check is applied automatically, at the level of a whole aggregate batch of tableaux, and any pair exceeding tolerance can be flagged and removed as a likely branch-jump artifact (Section 6.5). This is exactly the automated, large-scale counterpart of the by-hand check carried out in Section 7: there, d_H applied to $\{+\frac{1}{2\sqrt{3}}\}$ and $-\{-\frac{1}{2\sqrt{3}}\}$ vanishes identically, since the two single-element sets are literally equal; for shapes too large to solve in closed form, the same comparison is instead carried out numerically, at whatever precision the continuation was run at, on every flip-related pair in a batch.

Young diagram	Tableau	Max. root difference
$\{3, 2, 1\}$	$\{\{1, 2, 6\}, \{3, 4\}, \{5\}\}$	4.7×10^{-17}
$\{3, 2, 1\}$	$\{\{1, 2, 6\}, \{3, 5\}, \{4\}\}$	2.9×10^{-17}
$\{4, 2, 1\}$	$\{\{1, 2, 3, 4\}, \{5, 6\}, \{7\}\}$	1.4×10^{-14}

Table 2: Maximum one-to-one root differences (level by level) between freshly recomputed and saved reference solutions, all well below the 10^{-8} pass tolerance used for the comparison.

Table 2 reports a representative set of from-scratch reruns compared against saved reference solutions; all three cases agree to well within tolerance. This establishes agreement with previously computed (and themselves once machine-precision-validated) references to the stated tolerance; it does not, by itself, certify absolute accuracy at the full working precision used internally.

9.3 Performance profile of a single tableau

Table 3 reports a non-overlapping timing breakdown for one complete computation of a seven-box tableau, totaling 8.95 s, with recovered roots matching a saved reference to within 6.6×10^{-17} .

Category	Seconds	Share
Symbolic (Wolfram Language) construction & equation preparation	3.43	38.3%
Bertini path tracking	0.92	10.3%
Other back-end process overhead	1.04	11.6%
Kernel launch/load/finalization overhead	3.57	39.9%

Table 3: Timing breakdown for one small tableau. Bertini’s own path-tracking cost is a modest fraction of the total; fixed per-call kernel and symbolic-preparation overhead dominates at this problem size.

For this particular (small) example, fixed overhead — Wolfram Language kernel initialization and one-time symbolic-preparation cost — dominates the wall-clock time, while the genuine numerical continuation cost (Bertini path tracking) is comparatively small; this conclusion is specific to the profiled example and has not been established for substantially larger Young diagrams, where the polynomial degrees, and hence the intrinsic path-tracking cost, grow.

10 Discussion

10.1 Summary

We have given a formal account of the Q-system/Wronskian construction used to solve the (super-symmetric) nested Bethe ansatz equations for eigenstates labeled by standard Young tableaux: Q-functions built as Casoratian determinants of a shared linear difference problem obey conservation-law-type bilinear identities under the shift $u \rightarrow u \pm i$ (Eq. (7)), the discrete analogue of which, for the graded (bosonic/fermionic) elementary blocks of Section 3, generates the full Q-system by the Wronskian recursion (8)–(9); the marked-point freedom in splitting elementary building blocks into bosonic and fermionic families corresponds to a choice of Kac–Dynkin grading; a single chain inhomogeneity, continued from an asymptotically exact combinatorial solution down to the

physical homogeneous point, converts the remaining numerical work into a sequence of parameter homotopy continuations, formalized through the Davidenko equation and realized numerically by Bertini’s adaptive multiple-precision tracker; and an independent rigged-configuration/string-hypothesis combinatorics supplies both an approximate analytic cross-check and an exact combinatorial symmetry (the flip involution) used as an empirical validation criterion.

10.2 Limitations

The physical identity of the bosonic/fermionic split (Remark following Eq. 13) could not be settled from the codebase alone. The flip symmetry of Conjecture 5.1 is used, and validated numerically, but is not derived here from the representation theory of the underlying algebra. The branch-jump failure mode of Section 6.5 is mitigated by more conservative continuation settings but is not eliminated by any formal guarantee, and should continue to be screened for via the flip check on any newly computed batch of tableaux. Performance characterization to date covers only small tableaux; whether fixed overhead continues to dominate genuine path-tracking cost at large Young-diagram size remains open.

10.3 Outlook

Two directions follow directly from the results above. First, since the dominant cost observed in Table 3 is fixed per-call overhead rather than path-tracking time, reusing a single initialized symbolic-layer worker across many tableaux (rather than paying kernel startup once per tableau) is a natural, low-risk optimization for batch computations. Second, extending the performance profile of Section 9 to substantially larger Young diagrams would determine whether the numerical continuation itself, rather than fixed overhead, becomes the dominant cost once the $\mathbb{Q}$ -system’s polynomial degrees grow, which would in turn indicate where further numerical effort (tighter path-tracking control, higher base precision, or an adaptive step-size schedule tuned specifically to this problem’s observed branch-jump risk) is best directed.